**Faglig fornyelse – Hono Chat**

**Emne:** Backendudvikling med Hono, WebSockets, SQLite og TypeScript
**Uddannelse:** Webudvikler – Roskilde Tekniske Skole
**Projekt:** Hono Chat

**1. Indledning**

I forbindelse med faget Faglig fornyelse har jeg valgt at arbejde med Hono, som er et moderne webframework, der blandt andet kan bruges til backendudvikling med JavaScript og TypeScript.

Jeg blev introduceret til Hono af min lærer, som fortalte, at teknologien havde flere interessante muligheder. Det gjorde mig nysgerrig, og jeg besluttede derfor at undersøge, hvordan Hono kunne anvendes i praksis.

Mit mål var at udvikle en chatapplikation, hvor to personer kunne kommunikere med hinanden i realtid.

Projektet skulle samtidig give mig mulighed for at arbejde med flere forskellige teknologier og få en bedre forståelse af, hvordan frontend og backend kommunikerer.

Under udviklingen har jeg arbejdet med Hono, WebSockets, JavaScript, TypeScript, SQLite, HTML, Tailwind CSS, GitHub og Render.

**2. Hvad er Hono?**

Hono er et let og hurtigt webframework, som kan bruges til at udvikle webapplikationer og API'er.

Hono kan blandt andet anvendes sammen med Node.js og forskellige andre JavaScript-runtime-miljøer.

Jeg valgte Hono, fordi min lærer havde fortalt om dets muligheder, og fordi jeg gerne ville undersøge en teknologi, som jeg ikke tidligere havde arbejdet med.

Et alternativ til Hono er Express, som også er et populært framework til backendudvikling.

Hono er ikke nødvendigvis bedre end Express i alle situationer, men det har nogle interessante egenskaber. Frameworket er let, har god understøttelse af TypeScript og kan anvendes på flere forskellige platforme.

For en webudvikler betyder det, at man kan udvikle backendfunktionalitet uden at skulle opbygge hele serverstrukturen fra bunden.

I mit projekt brugte jeg Hono til at oprette en server, som kunne håndtere forbindelser mellem forskellige brugere.

**3. Projektets formål og funktionalitet**

Formålet med Hono Chat var at udvikle en applikation, hvor brugere kunne sende beskeder til hinanden.

En chatapplikation løser et praktisk problem, fordi den gør det muligt for mennesker at kommunikere digitalt, selvom de befinder sig forskellige steder.

Jeg ønskede derfor at udvikle en løsning, hvor en bruger kunne vælge en anden bruger og sende en besked, som blev leveret med det samme.

I applikationen har jeg arbejdet med forskellige brugere, blandt andet Fatih, Sarah, Adam og Vlad.

Brugerfladen indeholder en oversigt over kontakter, et område til visning af beskeder og et inputfelt, hvor brugeren kan skrive og sende en besked.

Jeg har også arbejdet med at gøre brugerfladen responsiv, så den kan anvendes på både computer og mobiltelefon.

Projektet er et eksempel på, hvordan man kan kombinere frontend og backend i én samlet webapplikation.

**4. WebSockets og kommunikation i realtid**

En af de vigtigste teknologier i mit projekt er WebSockets.

WebSockets gør det muligt at oprette en vedvarende forbindelse mellem klienten og serveren.

Ved almindelige HTTP-forespørgsler sender klienten en anmodning til serveren og modtager derefter et svar. Hvis klienten hele tiden skal undersøge, om der er kommet nye beskeder, kan man eksempelvis bruge gentagne HTTP-forespørgsler.

Med WebSockets kan serveren derimod sende nye beskeder til klienten gennem en allerede etableret forbindelse.

Det er særligt nyttigt i en chatapplikation, hvor brugerne forventer at modtage beskeder hurtigt.

I mit projekt oprettede jeg en WebSocket-forbindelse mellem browseren og Hono-serveren.

Når Fatih sender en besked til Vlad, bliver beskeden sendt til serveren gennem WebSocket-forbindelsen.

Serveren undersøger derefter, hvilken bruger der skal modtage beskeden, og forsøger at sende den videre til den pågældende brugers aktive forbindelse.

På den måde behøver browseren ikke konstant at hente nye beskeder gennem gentagne HTTP-forespørgsler.

Jeg har dermed fået en bedre forståelse af, hvordan realtidskommunikation kan implementeres i en webapplikation.

**5. JavaScript og TypeScript**

Jeg har anvendt JavaScript til at udvikle funktionaliteten i brugerfladen.

JavaScript bruges blandt andet til at registrere brugerens handlinger, vælge samtalepartnere, sende beskeder og opdatere indholdet på siden.

Når en bruger skriver en besked og trykker på send-knappen eller Enter, bliver der udført JavaScript-kode, som sender beskeden videre.

På backend har jeg arbejdet med TypeScript.

TypeScript bygger på JavaScript og giver mulighed for at anvende statiske typer, som kan hjælpe med at opdage bestemte fejl under udviklingen.

En vigtig erfaring fra projektet er, at man ikke behøver at bruge et helt andet programmeringssprog til backendudvikling.

JavaScript og TypeScript kan anvendes på både frontend og backend, eksempelvis ved hjælp af Node.js.

Det gør det muligt for en webudvikler at arbejde med store dele af en applikation inden for samme programmeringssprogsfamilie.

**6. SQLite og lagring af beskeder**

For at kunne gemme beskeder valgte jeg at arbejde med SQLite.

SQLite er en relationsdatabase, som gemmer sine data i en databasefil.

Jeg valgte SQLite, fordi teknologien var forholdsvis nem at komme i gang med, og fordi den kunne bruges direkte sammen med mit Node.js-projekt.

Jeg anvendte Node.js' indbyggede SQLite-modul.

Jeg oprettede en tabel med navnet messages, som indeholder oplysninger om afsender, modtager, besked og tidspunkt.

Formålet var at sikre, at beskeder kunne gemmes, så de ikke kun eksisterede i browseren, mens brugerne var online.

SQLite er velegnet til mindre projekter og prototyper, fordi man ikke behøver at installere og administrere en separat databaseserver.

En vigtig begrænsning er dog, at SQLite ikke automatisk løser alle problemer med datalagring i et produktionsmiljø. Man skal blandt andet tage hensyn til filplacering, adgangsrettigheder, sikkerhedskopiering og vedvarende lagring.

**7. Udvikling af brugerfladen**

Jeg har anvendt HTML til at opbygge strukturen i chatapplikationen og Tailwind CSS til at arbejde med design og layout.

Brugerfladen består blandt andet af en sidebar med kontakter, et chatområde og et felt til at skrive beskeder.

Med Tailwind CSS kunne jeg anvende utility-klasser direkte i HTML til at styre eksempelvis farver, afstande, bredder og placering af elementer.

Jeg arbejdede også med responsivt design.

På en computerskærm kan kontaktlisten og chatområdet vises ved siden af hinanden. På en mindre mobilskærm er det mere hensigtsmæssigt at skifte mellem kontaktlisten og den aktive samtale.

Under arbejdet oplevede jeg forskellige udfordringer med layoutet, blandt andet højden på chatområdet og placeringen af inputfeltet.

Disse udfordringer gav mig praktisk erfaring med CSS, Tailwind og manipulation af HTML-elementer gennem JavaScript.

**8. Udfordringer og fejlfinding**

Under udviklingen oplevede jeg flere tekniske udfordringer.

Nogle gange havde jeg svært ved at forstå, hvorfor koden ikke fungerede som forventet. Andre gange stødte jeg på kodeeksempler eller fremgangsmåder, som ikke passede til de versioner af teknologierne, jeg anvendte.

Jeg brugte blandt andet ChatGPT som hjælpemiddel til at forstå fejlmeddelelser, undersøge mulige løsninger og få forklaret kode.

Det var især nyttigt, når jeg havde brug for at forstå sammenhængen mellem frontend, backend og databasen.

En af de konkrete udfordringer opstod, da jeg skulle lægge applikationen online på Render.

Den første deployment mislykkedes, fordi applikationen ikke kunne åbne SQLite-databasefilen.

Problemet hang sammen med den filplacering, som applikationen forsøgte at anvende.

Jeg ændrede derfor databasekoden, så applikationen oprettede en mappe til databasen, inden SQLite-forbindelsen blev etableret.

Efter ændringen lykkedes det at deploye applikationen.

Jeg lærte samtidig, at databasefiler med private beskeder ikke bør lægges i et offentligt GitHub-repository.

Derfor anvendte jeg også .gitignore til at udelukke databasefiler fra versionsstyringen.

Denne oplevelse lærte mig, at en applikation godt kan fungere lokalt, men stadig kræve ændringer, før den fungerer i et hostingmiljø.

**9. GitHub og Render**

Jeg anvendte Git og GitHub til versionsstyring og til at gemme projektets kildekode online.

GitHub gjorde det muligt at dele koden med min lærer og samtidig forbinde projektet med Render.

Render er en cloudplatform, som blandt andet kan bruges til at hoste webapplikationer.

Jeg forbandt mit GitHub-repository med Render og konfigurerede applikationen til at blive bygget og startet som en Node.js-applikation.

Projektet anvender TypeScript, og derfor skal koden kompileres til JavaScript, inden den startes i produktionsmiljøet.

Jeg anvendte følgende kommandoer:

- npm install && npm run build til installation og kompilering.
- npm run start til at starte applikationen.

Efter at jeg havde rettet databaseproblemet, viste Render, at applikationen var live.

Jeg lærte dermed, hvordan man kan arbejde med en udviklingsproces, hvor ændringer først foretages lokalt, derefter sendes til GitHub og til sidst deployes til en server.

**10. Hvordan gør teknologierne webudviklerens hverdag nemmere?**

Hono, WebSockets, SQLite og TypeScript kan gøre udviklingsarbejdet lettere på forskellige måder.

Hono giver webudvikleren et framework til at oprette serverfunktionalitet og API'er uden at skulle udvikle alle grundlæggende funktioner selv.

WebSockets gør det lettere at implementere funktioner, hvor information skal udveksles hurtigt mellem klient og server. Det kan eksempelvis være chatapplikationer, live-notifikationer og samarbejdsværktøjer.

SQLite gør det muligt hurtigt at oprette en database til mindre applikationer uden at skulle konfigurere en separat databaseserver.

TypeScript kan forbedre kodekvaliteten ved at hjælpe udvikleren med at opdage bestemte typer fejl tidligere i processen.

Jeg har desuden erfaret, at AI-værktøjer som ChatGPT kan hjælpe med at forklare nye teknologier og give forslag til fejlfinding.

AI kan dog også foreslå kode, som ikke passer til projektets versioner eller struktur. Derfor er det vigtigt at kontrollere løsningerne og selv forsøge at forstå, hvordan de fungerer.

For mig har kombinationen af disse teknologier været en praktisk måde at lære nye områder af webudvikling på.

**11. Begrænsninger og mulige forbedringer**

Selvom projektet er blevet deployet, er der stadig områder, som kan undersøges og forbedres.

En væsentlig begrænsning er datalagringen på Render.

På en almindelig Render-webservice er det lokale filsystem ikke nødvendigvis permanent. Det betyder, at en SQLite-database kan gå tabt ved eksempelvis en genstart eller en ny deployment, hvis der ikke anvendes en løsning med vedvarende lagring.

En mere robust løsning kunne være en ekstern database eller en hostingløsning med persistent storage.

En anden mulig forbedring er brugerautentifikation, så man kan sikre, at en person kun kan sende beskeder som sin egen bruger.

Derudover kunne man arbejde videre med håndtering af afbrudte WebSocket-forbindelser, fejlhåndtering og sikkerhed.

Det vil også være relevant at teste, hvordan applikationen fungerer med flere samtidige brugere.

Disse forbedringer er ikke nødvendigvis en del af den oprindelige opgave, men de viser, hvilke overvejelser man skal gøre sig, hvis en prototype skal videreudvikles til en mere færdig applikation.

**12. Konklusion og faglig refleksion**

Arbejdet med Hono Chat har givet mig praktisk erfaring med backendudvikling og realtidskommunikation.

Da jeg begyndte projektet, var mit primære mål at udvikle en chatapplikation, hvor to personer kunne kommunikere med hinanden.

Undervejs blev projektet også en mulighed for at lære, hvordan flere forskellige teknologier arbejder sammen.

Jeg har fået erfaring med at oprette en backendserver med Hono, anvende WebSockets til kommunikation, arbejde med SQLite og bruge JavaScript og TypeScript i samme projekt.

Jeg har også lært mere om GitHub, deployment og de udfordringer, der kan opstå, når en applikation flyttes fra et lokalt udviklingsmiljø til en cloudserver.

En af de vigtigste erfaringer var, at backendudvikling ikke nødvendigvis kræver et andet programmeringssprog end det, man anvender på frontend.

Jeg har samtidig oplevet, at fejlfinding er en væsentlig del af webudviklerens arbejde.

Selvom jeg endnu ikke forstår alle tekniske detaljer fuldt ud, har projektet givet mig et bedre grundlag for at arbejde videre med backendteknologier.

Min største faglige læring er, at jeg gennem praktisk arbejde kan tilegne mig nye teknologier, undersøge problemer og gradvist udvikle min forståelse af, hvordan en moderne webapplikation fungerer.

Hono Chat har derfor været et relevant projekt i Faglig fornyelse, fordi det har givet mig mulighed for både at undersøge en ny teknologi, anvende den i praksis og reflektere over dens muligheder og begrænsninger.

---

**Projektlinks**

**GitHub:** https://github.com/fatihsolutions/hono-chat

**Liveapplikation:** https://hono-chat-9vig.onrender.com

**Teknologier anvendt**

Hono, Node.js, TypeScript, JavaScript, WebSockets, SQLite, HTML, Tailwind CSS, Git, GitHub og Render.